

Mônadas

uma exposição matemático-funcional

Arthur P. de Aguiar Pereira

Universidade Estadual de Ponta Grossa



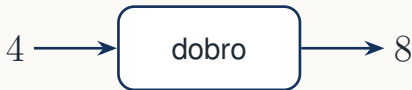
Uma função é uma máquina

Na matemática e na programação funcional, uma função é uma regra: **mesma entrada, mesma saída**, e nada mais acontece.



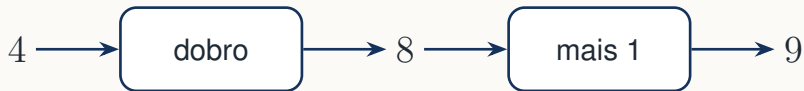
Uma função é uma máquina

Na matemática e na programação funcional, uma função é uma regra: **mesma entrada, mesma saída**, e nada mais acontece.



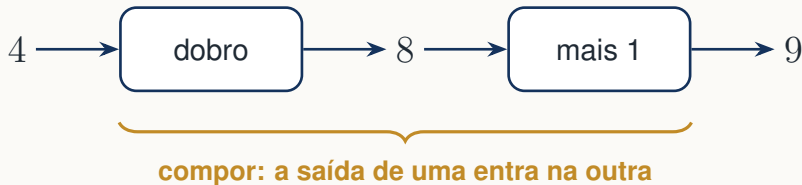
Uma função é uma máquina

Na matemática e na programação funcional, uma função é uma regra: **mesma entrada, mesma saída**, e nada mais acontece.



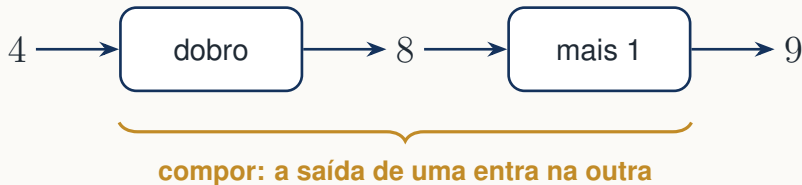
Uma função é uma máquina

Na matemática e na programação funcional, uma função é uma regra: **mesma entrada, mesma saída**, e nada mais acontece.



Uma função é uma máquina

Na matemática e na programação funcional, uma função é uma regra: **mesma entrada, mesma saída**, e nada mais acontece.



Programas grandes nascem assim: **peças pequenas, encaixadas**.
Programar só com funções desse tipo é a **programação funcional**.

Mas programas de verdade fazem mais do que calcular

Podem falhar

dividir por zero,
raiz de número negativo

Mas programas de verdade fazem mais do que calcular

Podem falhar

dividir por zero,
raiz de número negativo

Guardam estado

um contador,
o saldo de uma conta

Mas programas de verdade fazem mais do que calcular

Podem falhar

dividir por zero,
raiz de número negativo

Guardam estado

um contador,
o saldo de uma conta

Falam com o mundo

ler um arquivo,
imprimir na tela

Mas programas de verdade fazem mais do que calcular

Podem falhar

dividir por zero,
raiz de número negativo

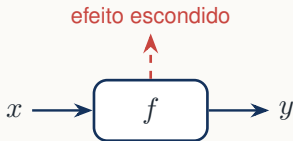
Guardam estado

um contador,
o saldo de uma conta

Falam com o mundo

ler um arquivo,
imprimir na tela

Isso se chama **efeito**.



numa linguagem comum

Mas programas de verdade fazem mais do que calcular

Podem falhar

dividir por zero,
raiz de número negativo

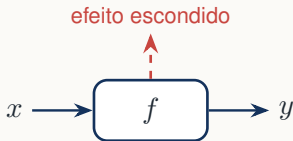
Guardam estado

um contador,
o saldo de uma conta

Falam com o mundo

ler um arquivo,
imprimir na tela

Isso se chama **efeito**.



numa linguagem comum



na programação funcional: o efeito vai **para a saída**

A pergunta desta apresentação

Como continuar **compondo** funções
quando elas têm **efeitos**?

A pergunta desta apresentação

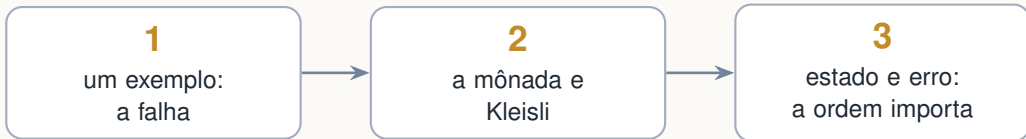
Como continuar **compondo** funções
quando elas têm **efeitos**?

Resposta: **mônadas**, uma estrutura da teoria das categorias (anos 1950–60).
Moggi (1989) a usou para modelar computações; Wadler (1992), para programar.

A pergunta desta apresentação

Como continuar **compondo** funções quando elas têm **efeitos**?

Resposta: **mônadas**, uma estrutura da teoria das categorias (anos 1950–60).
Moggi (1989) a usou para modelar computações; Wadler (1992), para programar.



Quando a função pode falhar, a composição quebra

Duas funções que podem **falhar**:

$$\text{inv}(x) = \frac{1}{x} \qquad \text{raiz}(x) = \sqrt{x}$$

$\text{inv}(0)$ e $\text{raiz}(-2)$ **não existem**.

Quando a função pode falhar, a composição quebra

Duas funções que podem **falhar**:

$$\text{inv}(x) = \frac{1}{x} \qquad \text{raiz}(x) = \sqrt{x}$$

$\text{inv}(0)$ e $\text{raiz}(-2)$ **não existem**.

Solução: a resposta diz se **deu certo** ou se deu **erro**.

$$\text{inv}(4) = \text{ok}(0,25) \qquad \text{inv}(0) = \text{erro}$$

A saída agora é “**número ou erro**”: o tipo $\mathbb{R} + E$.
(**ok** e **erro** são as injeções ι_1 e ι_2 de $\mathbb{R} + E$.)

Quando a função pode falhar, a composição quebra

Duas funções que podem **falhar**:

$$\text{inv}(x) = \frac{1}{x} \quad \text{raiz}(x) = \sqrt{x}$$

$\text{inv}(0)$ e $\text{raiz}(-2)$ **não existem**.

Solução: a resposta diz se **deu certo** ou se deu **erro**.

$$\text{inv}(4) = \text{ok}(0,25) \quad \text{inv}(0) = \text{erro}$$

A saída agora é “**número ou erro**”: o tipo $\mathbb{R} + E$.
(**ok** e **erro** são as injeções ι_1 e ι_2 de $\mathbb{R} + E$.)

$$\mathbb{R} \xrightarrow{\text{inv}} \mathbb{R} + E \xrightarrow{\text{raiz ?}} \mathbb{R} + E$$

raiz espera um **número**, mas recebe “número ou erro”.

Estender a função resolve, e o gesto é sempre o mesmo

$$\text{raiz}^*(\text{ok}(y)) = \text{raiz}(y) \qquad \text{raiz}^*(\text{erro}) = \text{erro}$$

raiz^* (“raiz estrela”): a raiz **estendida**, que aceita “número ou erro”.

Estender a função resolve, e o gesto é sempre o mesmo

$$\text{raiz}^*(\text{ok}(y)) = \text{raiz}(y) \qquad \text{raiz}^*(\text{erro}) = \text{erro}$$

raiz^* (“raiz estrela”): a raiz **estendida**, que aceita “número ou erro”.

Agora encaixa: $x \xrightarrow{\text{inv}} \cdot \xrightarrow{\text{raiz}^*} \cdot$

x	$\text{inv}(x)$	$\text{raiz}^*(\text{inv}(x))$
4	$\text{ok}(0,25)$	$\text{ok}(0,5)$
0	erro	erro
-2	$\text{ok}(-0,5)$	erro

Estender a função resolve, e o gesto é sempre o mesmo

$$\text{raiz}^*(\text{ok}(y)) = \text{raiz}(y) \qquad \text{raiz}^*(\text{erro}) = \text{erro}$$

raiz^* (“raiz estrela”): a raiz **estendida**, que aceita “número ou erro”.

Agora encaixa: $x \xrightarrow{\text{inv}} \cdot \xrightarrow{\text{raiz}^*} \cdot$

x	$\text{inv}(x)$	$\text{raiz}^*(\text{inv}(x))$
4	$\text{ok}(0,25)$	$\text{ok}(0,5)$
0	erro	erro
-2	$\text{ok}(-0,5)$	erro

O gesto é sempre o mesmo, para qualquer função: **se deu erro, repassa; se deu certo, continua.**

Embrulhar como sucesso, $\eta(x) = \text{ok}(x)$, não muda nada. **Mônada** = esse padrão, com nome.

Mônada: uma caixa, embrulhar e juntar

Uma **mônada** são três peças:

- T , a **caixa**: leva um tipo A ao tipo TA das *computações* que produzem um A .
Ex.: $TA = A + E$ (“ A ou erro”); TA = listas de A .

Mônada: uma caixa, embrulhar e juntar

Uma **mônada** são três peças:

- T , a **caixa**: leva um tipo A ao tipo TA das *computações* que produzem um A .
Ex.: $TA = A + E$ (“ A ou erro”); TA = listas de A .
- η , **embrulhar**: põe um valor na caixa, sem efeito.
Ex.: $\eta(4) = \text{ok}(4)$; $\eta(3) = [3]$.

Mônada: uma caixa, embrulhar e juntar

Uma **mônada** são três peças:

- T , a **caixa**: leva um tipo A ao tipo TA das *computações* que produzem um A .
Ex.: $TA = A + E$ (“ A ou erro”); TA = listas de A .
- η , **embrulhar**: põe um valor na caixa, sem efeito.
Ex.: $\eta(4) = \text{ok}(4)$; $\eta(3) = [3]$.
- μ , **juntar**: caixa dentro de caixa vira uma caixa só.
Ex.: $\mu(\text{ok}(\text{ok}(4))) = \text{ok}(4)$; $\mu([[1, 2], [3]]) = [1, 2, 3]$.

Duas regras, como o 0 na soma: **juntar não depende da ordem; embrulhar e juntar não muda nada.**

Formalmente: $T : \mathcal{C} \rightarrow \mathcal{C}$ endofuntor, $\eta : \text{Id} \Rightarrow T$ e $\mu : T^2 \Rightarrow T$ transformações naturais, com $\mu_A \circ \mu_{TA} = \mu_A \circ T(\mu_A)$ e $\mu_A \circ \eta_{TA} = \text{id}_{TA} = \mu_A \circ T(\eta_A)$.

As leis: compor não depende do agrupamento

$$\begin{array}{ccc} T^3 A & \xrightarrow{\mu_{TA}} & T^2 A \\ T(\mu_A) \downarrow & & \downarrow \mu_A \\ T^2 A & \xrightarrow{\mu_A} & TA \end{array}$$

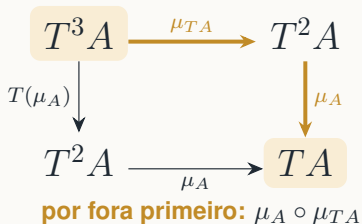
Com listas, passo a passo

Exemplo: $x = [[[1, 2], [3]], [[4]]] \in T^3 A$, uma lista de listas de listas: **três níveis de colchetes**.

Nas listas, μ é **achatar**: juntar duas camadas é apagar um nível de colchetes.

Leia assim Cada seta é uma função; seguir setas é **compor**. O quadrado **comuta**: os dois caminhos de $T^3 A$ até TA dão o mesmo resultado. $T^3 A =$ três camadas, $T(T(TA))$.

As leis: compor não depende do agrupamento



Com listas, passo a passo

$$x = [[[1, 2], [3]], [[4]]] \xrightarrow{\mu_{TA}} [[1, 2], [3], [4]] \xrightarrow{\mu_A} [1, 2, 3, 4]$$

μ_{TA} emenda $[1, 2], [3]$ com $[4]$: as duas camadas de fora viram uma. Depois μ_A achata o resto.

Leia assim μ_{TA} funde as duas camadas **de fora**; $T(\mu_A)$ entra e funde as duas **de dentro**. Colchetes coloridos: os que a **próxima seta apaga**.

As leis: compor não depende do agrupamento

$$\begin{array}{ccc} T^3 A & \xrightarrow{\mu_{TA}} & T^2 A \\ \downarrow T(\mu_A) & & \downarrow \mu_A \\ T^2 A & \xrightarrow{\mu_A} & TA \end{array}$$

por dentro primeiro: $\mu_A \circ T(\mu_A)$

Com listas, passo a passo

$$x = [[[1, 2], [3]], [[4]]] \xrightarrow{T(\mu_A)} [[1, 2, 3], [4]] \xrightarrow{\mu_A} [1, 2, 3, 4]$$

$T(\mu_A)$ achata **dentro de cada elemento**: $[[1, 2], [3]] \mapsto [1, 2, 3]$ e $[[4]] \mapsto [4]$; depois μ_A .

Leia assim μ_{TA} funde as duas camadas **de fora**; $T(\mu_A)$ entra e funde as duas **de dentro**. Colchetes coloridos: os que a **próxima seta apaga**.

As leis: compor não depende do agrupamento

$$\begin{array}{ccc} T^3 A & \xrightarrow{\mu_{TA}} & T^2 A \\ \downarrow T(\mu_A) & & \downarrow \mu_A \\ T^2 A & \xrightarrow{\mu_A} & T A \end{array}$$
$$\mu_A \circ \mu_{TA} = \mu_A \circ T(\mu_A)$$

Com listas, passo a passo

dourado: $x \xrightarrow{\mu_{TA}} [[1, 2], [3], [4]] \xrightarrow{\mu_A} [1, 2, 3, 4]$

azul: $x \xrightarrow{T(\mu_A)} [[1, 2, 3], [4]] \xrightarrow{\mu_A} [1, 2, 3, 4]$ mesmo resultado.

Leia assim μ_{TA} funde as duas camadas **de fora**; $T(\mu_A)$ entra e funde as duas **de dentro**. Colchetes coloridos: os que a **próxima seta apaga**.

As leis: compor não depende do agrupamento

$$\begin{array}{ccc} T^3 A & \xrightarrow{\mu_{TA}} & T^2 A \\ T(\mu_A) \downarrow & & \downarrow \mu_A \\ T^2 A & \xrightarrow{\mu_A} & TA \\ \mu_A \circ \mu_{TA} & = & \mu_A \circ T(\mu_A) \end{array}$$

$$\begin{array}{ccccc} TA & \xrightarrow{\eta_{TA}} & T^2 A & \xleftarrow{T(\eta_A)} & TA \\ & \searrow \text{id}_{TA} & \downarrow \mu_A & & \swarrow \text{id}_{TA} \\ & & TA & & \end{array}$$

uma camada a mais, por fora ou por dentro...

Com listas, passo a passo

Exemplo: $m = [1, 2] \in TA$. Aqui η põe um valor sozinho numa lista, $\eta(1) = [1]$.

Vamos acrescentar uma camada com η e depois juntar com μ_A .

Leia assim η_{TA} embrulha **por fora**; $T(\eta_A)$ embrulha **por dentro**; id_{TA} = não fazer nada. Os triângulos dizem: embrulhar e juntar devolve o que se tinha.

As leis: compor não depende do agrupamento

$$\begin{array}{ccc} T^3 A & \xrightarrow{\mu_{TA}} & T^2 A \\ T(\mu_A) \downarrow & & \downarrow \mu_A \\ T^2 A & \xrightarrow{\mu_A} & TA \\ \mu_A \circ \mu_{TA} & = & \mu_A \circ T(\mu_A) \end{array}$$

$$\begin{array}{ccccc} TA & \xrightarrow{\eta_{TA}} & T^2 A & \xleftarrow{T(\eta_A)} & TA \\ & \searrow \text{id}_{TA} & \downarrow \mu_A & & \swarrow \text{id}_{TA} \\ & & TA & & \end{array}$$

por fora: $\mu_A \circ \eta_{TA} = \text{id}_{TA}$

Com listas, passo a passo

$$m = [1, 2] \xrightarrow{\eta_{TA}} [[1, 2]] \xrightarrow{\mu_A} [1, 2]$$

η_{TA} põe a lista inteira dentro de uma lista nova; μ_A achata e devolve m .

Leia assim η_{TA} embrulha **por fora**; $T(\eta_A)$ embrulha **por dentro**; id_{TA} = não fazer nada. Os triângulos dizem: embrulhar e juntar devolve o que se tinha.

As leis: compor não depende do agrupamento

$$\begin{array}{ccc} T^3 A & \xrightarrow{\mu_{TA}} & T^2 A \\ T(\mu_A) \downarrow & & \downarrow \mu_A \\ T^2 A & \xrightarrow{\mu_A} & TA \\ \mu_A \circ \mu_{TA} & = & \mu_A \circ T(\mu_A) \end{array}$$

$$\begin{array}{ccccc} TA & \xrightarrow{\eta_{TA}} & T^2 A & \xleftarrow{T(\eta_A)} & TA \\ & \searrow \text{id}_{TA} & \downarrow \mu_A & & \swarrow \text{id}_{TA} \\ & & TA & & \end{array}$$

por dentro: $\mu_A \circ T(\eta_A) = \text{id}_{TA}$; η é neutra

Com listas, passo a passo

dourado: $[1, 2] \xrightarrow{\eta_{TA}} [[1, 2]] \xrightarrow{\mu_A} [1, 2]$

azul: $[1, 2] \xrightarrow{T(\eta_A)} [[1], [2]] \xrightarrow{\mu_A} [1, 2]$ $T(\eta_A)$ embrulha **cada elemento**.

Leia assim η_{TA} embrulha **por fora**; $T(\eta_A)$ embrulha **por dentro**; id_{TA} = não fazer nada. Os triângulos dizem: embrulhar e juntar devolve o que se tinha.

Kleisli: compor funções que devolvem computações

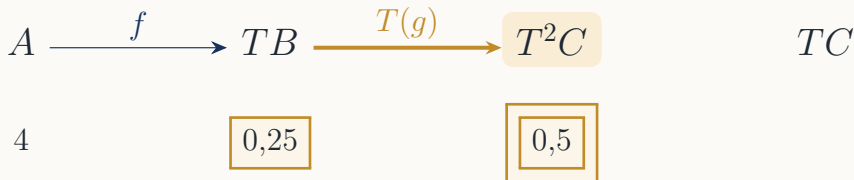
$f : A \rightarrow TB$ devolve um TB , mas $g : B \rightarrow TC$ espera um B . **Como compor?**



Leia assim f é uma **função com efeito**: recebe um valor e devolve uma **caixa**. Cada caixa dourada é uma camada. Exemplo da calculadora: $f = \text{inv}$, $g = \text{raiz}$, $x = 4$.

Kleisli: compor funções que devolvem computações

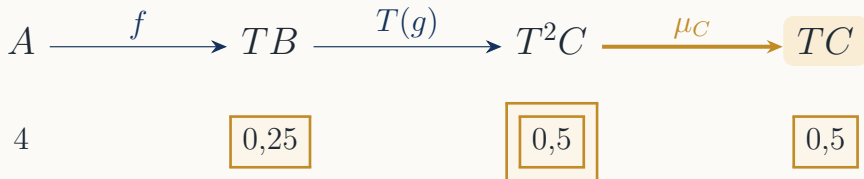
$f : A \rightarrow TB$ devolve um TB , mas $g : B \rightarrow TC$ espera um B . **Como compor?**



Leia assim $T(g)$ aplica g **dentro da caixa**. Como g também devolve uma caixa, ficam **duas camadas**.

Kleisli: compor funções que devolvem computações

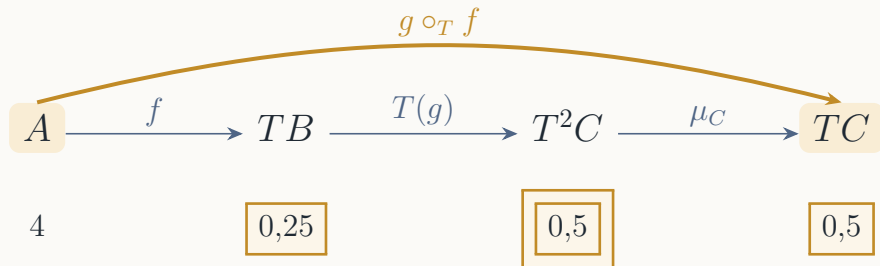
$f : A \rightarrow TB$ devolve um TB , mas $g : B \rightarrow TC$ espera um B . **Como compor?**



Leia assim μ_C **tira uma camada**: junta as duas caixas numa só. Chegamos em TC .

Kleisli: compor funções que devolvem computações

$f : A \rightarrow TB$ devolve um TB , mas $g : B \rightarrow TC$ espera um B . **Como compor?**

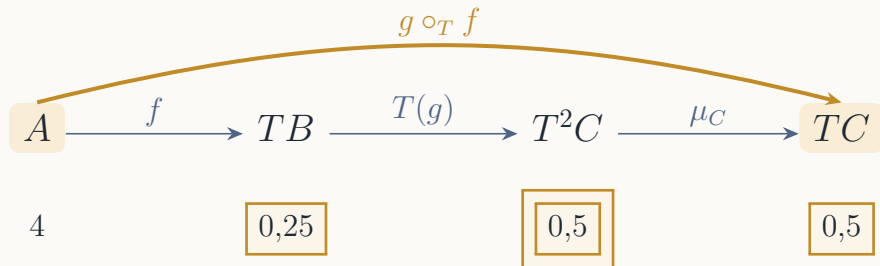


$$g \circ_T f := \mu_C \circ T(g) \circ f \quad (\text{composição de Kleisli})$$

Leia assim A seta curva não é um caminho novo: é **um nome** para o caminho de baixo inteiro. Primeiro f , depois g .

Kleisli: compor funções que devolvem computações

$f : A \rightarrow TB$ devolve um TB , mas $g : B \rightarrow TC$ espera um B . **Como compor?**



$$g \circ_T f = g^* \circ f, \quad g^* := \mu_C \circ T(g) \quad (\text{extensão de Kleisli})$$

Leia assim g^* (“ g estrela”) é g adaptada para receber uma caixa; no exemplo, é o raiz^* do início. Na programação: $m \gg g := g^*(m)$, “rode m e passe o resultado a g ”.

Duas mônadas: erro e estado

Erro: “resultado ou erro”

$$\eta(x) = \text{ok}(x)$$

μ junta dois níveis:

$$\text{ok}(\text{ok}(x)) \mapsto \text{ok}(x)$$

$$\text{ok}(\text{erro}) \mapsto \text{erro}$$

$$\text{erro} \mapsto \text{erro}$$

um erro em qualquer nível **vence**

Estado: “lê e muda um painel”

computação = função

$$\text{estado} \longmapsto (\text{valor}, \text{estado novo})$$

$\eta(x)$: devolve x , não mexe no estado

μ : roda a primeira etapa e **passa o estado novo** para a segunda

Duas mônadas: erro e estado

Erro: “resultado ou erro”

$$\eta(x) = \text{ok}(x)$$

μ junta dois níveis:

$$\text{ok}(\text{ok}(x)) \mapsto \text{ok}(x)$$

$$\text{ok}(\text{erro}) \mapsto \text{erro}$$

$$\text{erro} \mapsto \text{erro}$$

um erro em qualquer nível **vence**

Estado: “lê e muda um painel”

computação = função

$$\text{estado} \mapsto (\text{valor}, \text{estado novo})$$

$\eta(x)$: devolve x , não mexe no estado

μ : roda a primeira etapa e **passa o estado novo** para a segunda

Contador: $\text{prox}(n) = (n, n + 1)$, devolve o número atual e soma 1.

$s = 0$

prox

$s = 1$

prox

$s = 2$

Duas mônadas: erro e estado

Erro: “resultado ou erro”

$$\eta(x) = \text{ok}(x)$$

μ junta dois níveis:

$$\text{ok}(\text{ok}(x)) \mapsto \text{ok}(x)$$

$$\text{ok}(\text{erro}) \mapsto \text{erro}$$

$$\text{erro} \mapsto \text{erro}$$

um erro em qualquer nível **vence**

Estado: “lê e muda um painel”

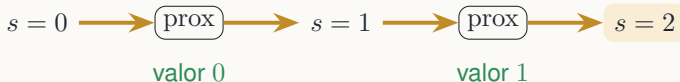
computação = função

$$\text{estado} \mapsto (\text{valor}, \text{estado novo})$$

$\eta(x)$: devolve x , não mexe no estado

μ : roda a primeira etapa e **passa o estado novo** para a segunda

Contador: $\text{prox}(n) = (n, n + 1)$, devolve o número atual e soma 1.



Duas mônadas: erro e estado

Erro: “resultado ou erro”

$$\eta(x) = \text{ok}(x)$$

μ junta dois níveis:

$$\text{ok}(\text{ok}(x)) \mapsto \text{ok}(x)$$

$$\text{ok}(\text{erro}) \mapsto \text{erro}$$

$$\text{erro} \mapsto \text{erro}$$

um erro em qualquer nível **vence**

Estado: “lê e muda um painel”

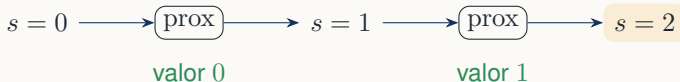
computação = função

$$\text{estado} \mapsto (\text{valor}, \text{estado novo})$$

$\eta(x)$: devolve x , não mexe no estado

μ : roda a primeira etapa e **passa o estado novo** para a segunda

Contador: $\text{prox}(n) = (n, n + 1)$, devolve o número atual e soma 1.



Nenhuma etapa fala do estado: quem o passa adiante é μ .

Juntar dois efeitos: trocar as camadas do meio

Queremos **estado** e **falha**: combinar duas mônadas T e U .

Leia assim $T = \text{caixa}$, $U = \text{envelope}$; leia as letras **de dentro para fora**: $TUA = T(U(A))$. Cada mônada só junta embalagens *iguais e encostadas*: μ^T caixa com caixa, μ^U envelope com envelope.

Exemplo concreto: uma loja. $T = \text{lista}$ (vários pedidos); $U = \text{exceção}$, com $\text{ok} = \iota_1$ e $\text{erro} = \iota_2$. Chegam dois lotes:

$$x = [\text{ok}([\text{ok}(\text{livro}), \text{erro}(\text{estoque})]), \text{erro}(\text{cartão})]$$

Lote 1 aceito, com um pedido ok e um sem estoque; lote 2 recusado (cartão). De fora para dentro: lista, ok/erro, lista, ok/erro: $x \in TUTUA$.

Juntar dois efeitos: trocar as camadas do meio

Queremos **estado** e **falha**: combinar duas mônadas T e U .

As camadas ficam intercaladas, $TUTU$. Para juntar, é preciso **trocar** o par do meio, UT por TU : essa regra de troca, λ , se chama *lei distributiva*.

Leia assim λ **troca o envelope e a caixa do meio** ($UT \rightsquigarrow TU$); depois cada mônada junta as suas. Em x , o par fora de ordem é o lote aceito, um ok *por fora* de uma lista:

$$x = [\text{ok}([\text{ok}(\text{livro}), \text{erro}(\text{estoque})]), \text{erro}(\text{cartão})]$$

Na loja, λ faz: $\text{ok}([a, b]) \mapsto [\text{ok}(a), \text{ok}(b)]$ e $\text{erro}(e) \mapsto [\text{erro}(e)]$.

λ é **um nome** (de uma transformação natural), não a função anônima $\lambda x. e$. **Compatível** = trocar não atrapalha embrulhar (η) nem juntar (μ): condições (i)–(iv).

Juntar dois efeitos: trocar as camadas do meio

Queremos **estado** e **falha**: combinar duas mônadas T e U .

As camadas ficam intercaladas, $TUTU$. Para juntar, é preciso **trocar** o par do meio, UT por TU : essa regra de troca, λ , se chama *lei distributiva*.

$$TUTUA \xrightarrow{T(\lambda_{UA})} TTUUA \xrightarrow{\mu_{U^2A}^T} TUUA \xrightarrow{T(\mu_A^U)} TUA$$

Leia assim Passo a passo na loja, a partir de $x = [\text{ok}([\text{ok}(\text{livro}), \text{erro}(\text{estoque})]), \text{erro}(\text{cartão})]$:

$T(\lambda_{UA})$ $[[\text{ok}(\text{ok}(\text{livro})), \text{ok}(\text{erro}(\text{estoque}))], [\text{erro}(\text{cartão})]]$ em cada lote, λ troca

O lote aceito vira a lista dos seus pedidos, cada um com um ok a mais por fora; o recusado vira uma lista com um erro só.

Juntar dois efeitos: trocar as camadas do meio

Queremos **estado** e **falha**: combinar duas mônadas T e U .

As camadas ficam intercaladas, $TUTU$. Para juntar, é preciso **trocar** o par do meio, UT por TU : essa regra de troca, λ , se chama *lei distributiva*.

$$TUTUA \xrightarrow{T(\lambda_{UA})} TTUUA \xrightarrow{\mu_{U^2A}^T} TUUA \xrightarrow{T(\mu_A^U)} TUA$$

Leia assim Passo a passo na loja, a partir de $x = [\text{ok}([\text{ok}(\text{livro}), \text{erro}(\text{estoque})]), \text{erro}(\text{cartão})]$:

$T(\lambda_{UA})$ $[[\text{ok}(\text{ok}(\text{livro})), \text{ok}(\text{erro}(\text{estoque}))], [\text{erro}(\text{cartão})]]$ em cada lote, λ troca

μ^T $[\text{ok}(\text{ok}(\text{livro})), \text{ok}(\text{erro}(\text{estoque})), \text{erro}(\text{cartão})]$ achata: uma lista só

Agora as duas listas estão encostadas, e μ^T (achatar) as junta: um item por pedido, cada um com dois níveis de ok/erro.

Juntar dois efeitos: trocar as camadas do meio

Queremos **estado** e **falha**: combinar duas mônadas T e U .

As camadas ficam intercaladas, $TUTU$. Para juntar, é preciso **trocar** o par do meio, UT por TU : essa regra de troca, λ , se chama *lei distributiva*.

$$TUTUA \xrightarrow{T(\lambda_{UA})} TTUUA \xrightarrow{\mu_{U^2A}^T} TUU A \xrightarrow{T(\mu_A^U)} TUA$$

Leia assim Passo a passo na loja, a partir de $x = [\text{ok}([\text{ok}(\text{livro}), \text{erro}(\text{estoque})]), \text{erro}(\text{cartão})]$:

$T(\lambda_{UA})$ $[[\text{ok}(\text{ok}(\text{livro})), \text{ok}(\text{erro}(\text{estoque}))], [\text{erro}(\text{cartão})]]$ em cada lote, λ troca

μ^T $[\text{ok}(\text{ok}(\text{livro})), \text{ok}(\text{erro}(\text{estoque})), \text{erro}(\text{cartão})]$ achata: uma lista só

$T(\mu^U)$ $[\text{ok}(\text{livro}), \text{erro}(\text{estoque}), \text{erro}(\text{cartão})]$ **relatório final**

μ^U junta os dois níveis em cada item: $\text{ok}(z) \mapsto z$ e $\text{erro}(e) \mapsto \text{erro}(e)$. Um item por pedido, com o motivo de cada erro.

Juntar dois efeitos: trocar as camadas do meio

Queremos **estado** e **falha**: combinar duas mônadas T e U .

As camadas ficam intercaladas, $TUTU$. Para juntar, é preciso **trocar** o par do meio, UT por TU : essa regra de troca, λ , se chama *lei distributiva*.

$$TUTUA \xrightarrow{T(\lambda_{UA})} TTUUA \xrightarrow{\mu_{U^2A}^T} TUUA \xrightarrow{T(\mu_A^U)} TUA$$

Resultado

O erro **sempre** troca de lugar assim, com *qualquer* mônada. Com o estado, o erro fica **dentro**: $((A + E) \times S)^S$, lê o estado e devolve (ok ou erro, estado novo).

Leia assim É a troca da loja, em geral: “ok de uma computação” vira a computação com cada resultado marcado ok; “erro” vira a computação que só devolve o erro. Em fórmula (artigo): $\lambda_A = [T(\iota_1), \eta^T \circ \iota_2]$.

Juntar dois efeitos: trocar as camadas do meio

Queremos **estado** e **falha**: combinar duas mônadas T e U .

As camadas ficam intercaladas, $TUTU$. Para juntar, é preciso **trocar** o par do meio, UT por TU : essa regra de troca, λ , se chama *lei distributiva*.

$$TUTUA \xrightarrow{T(\lambda_{UA})} TTUUA \xrightarrow{\mu_{U^2A}^T} TUUA \xrightarrow{T(\mu_A^U)} TUA$$

Resultado

O erro **sempre** troca de lugar assim, com *qualquer* mônada. Com o estado, o erro fica **dentro**: $((A + E) \times S)^S$, lê o estado e devolve (ok ou erro, estado novo).

Leia assim E no sentido contrário, $(A \times S)^S + E$? Aí é *ou* erro antes de tudo, *ou* computação de estado: o erro ficaria *fora* da função e **não poderia olhar o estado**...

Na outra ordem, não dá: o estado não distribui sobre o erro

Resultado. Se há pelo menos **dois estados** e **algum erro**, não existe lei distributiva que ponha o erro **por fora** do estado.

Erro dentro $((A+E) \times S)^S$:
lê o estado, *depois* decide se falha.

A troca teria de decidir, **sem olhar o estado**, se w vira erro:

Erro fora $(A \times S)^S + E$:
decide se falha *antes* de ler o estado.

w : lê o painel
em s_0 : **dá certo**
em s_1 : **dá erro**

Na outra ordem, não dá: o estado não distribui sobre o erro

Resultado. Se há pelo menos **dois estados** e **algum erro**, não existe lei distributiva que ponha o erro **por fora** do estado.

Erro dentro $((A+E) \times S)^S$:
lê o estado, *depois* decide se falha.

Erro fora $(A \times S)^S + E$:
decide se falha *antes* de ler o estado.

A troca teria de decidir, **sem olhar o estado**, se w vira erro:

Opção 1: “é erro”. Mas com o painel em s_0 , w **dá certo**.
Errado.

w : lê o painel

em s_0 : **dá certo**

em s_1 : **dá erro**

Na outra ordem, não dá: o estado não distribui sobre o erro

Resultado. Se há pelo menos **dois estados** e **algum erro**, não existe lei distributiva que ponha o erro **por fora** do estado.

Erro dentro $((A+E) \times S)^S$:
lê o estado, *depois* decide se falha.

Erro fora $(A \times S)^S + E$:
decide se falha *antes* de ler o estado.

w : lê o painel
em s_0 : **dá certo**
em s_1 : **dá erro**

A troca teria de decidir, **sem olhar o estado**, se w vira erro:

Opção 1: “é erro”. Mas com o painel em s_0 , w **dá certo**.
Errado.

Opção 2: “não é erro”. Mas com o painel em s_1 , w **falha**.
Errado.

Na outra ordem, não dá: o estado não distribui sobre o erro

Resultado. Se há pelo menos **dois estados** e **algum erro**, não existe lei distributiva que ponha o erro **por fora** do estado.

Erro dentro $((A+E) \times S)^S$:
lê o estado, *depois* decide se falha.

Erro fora $(A \times S)^S + E$:
decide se falha *antes* de ler o estado.

w : lê o painel
em s_0 : **dá certo**
em s_1 : **dá erro**

A troca teria de decidir, **sem olhar o estado**, se w vira erro:

Opção 1: “é erro”. Mas com o painel em s_0 , w **dá certo**.
Errado.

Opção 2: “não é erro”. Mas com o painel em s_1 , w **falha**.
Errado.

Nenhuma escolha serve: falhar depende do estado, mas o erro de fora não vê o estado. **Logo, a troca não existe.**

O que levar desta apresentação

- **Mônada** = caixa (T) + embrulhar (η) + juntar (μ). Com ela, funções com efeito **voltam a se compor** (Kleisli).

O que levar desta apresentação

- **Mônada** = caixa (T) + embrulhar (η) + juntar (μ). Com ela, funções com efeito **voltam a se compor** (Kleisli).
- É o μ que **decide como os efeitos se combinam**: o erro vence; o estado passa adiante.

O que levar desta apresentação

- **Mônada** = caixa (T) + embrulhar (η) + juntar (μ). Com ela, funções com efeito **voltam a se compor** (Kleisli).
- É o μ que **decide como os efeitos se combinam**: o erro vence; o estado passa adiante.
- **Estado e erro não comutam**: com o erro *dentro* do estado, a combinação existe; com o erro *fora*, não existe lei distributiva. **A ordem importa.**

As leis do *bind* são os axiomas de uma categoria

lei do <i>bind</i>	em Set	axioma de $\mathcal{C}_T$
$f^* \circ \eta_A = f$	$\eta(x) \gg f = f(x)$	identidade à direita
$(\eta_A)^* = \text{id}_{TA}$	$m \gg \eta = m$	identidade à esquerda
$(g^* \circ f)^* = g^* \circ f^*$	$(m \gg f) \gg g = m \gg (\lambda x. f(x) \gg g)$	associatividade de $\circ_T$

Leia assim As leis são as da soma, $a + 0 = a$, $0 + a = a$, $(a + b) + c = a + (b + c)$, com η no papel do 0.

Categoria = setas que se compõem, com identidades, de forma associativa.

As leis do *bind* são os axiomas de uma categoria

lei do <i>bind</i>	em Set	axioma de $\mathcal{C}_T$
$f^* \circ \eta_A = f$	$\eta(x) \gg f = f(x)$	identidade à direita
$(\eta_A)^* = \text{id}_{TA}$	$m \gg \eta = m$	identidade à esquerda
$(g^* \circ f)^* = g^* \circ f^*$	$(m \gg f) \gg g = m \gg (\lambda x. f(x) \gg g)$	associatividade de $\circ_T$

Leia assim As leis são as da soma, $a + 0 = a$, $0 + a = a$, $(a + b) + c = a + (b + c)$, com η no papel do 0.

Categoria = setas que se compõem, com identidades, de forma associativa.

Proposição 1

Toda mônada satisfaz essas leis: elas *decorrem* da funtorialidade de T e da naturalidade e coerência de η, μ .

As leis do *bind* são os axiomas de uma categoria

lei do <i>bind</i>	em Set	axioma de $\mathcal{C}_T$
$f^* \circ \eta_A = f$	$\eta(x) \gg f = f(x)$	identidade à direita
$(\eta_A)^* = \text{id}_{TA}$	$m \gg \eta = m$	identidade à esquerda
$(g^* \circ f)^* = g^* \circ f^*$	$(m \gg f) \gg g = m \gg (\lambda x. f(x) \gg g)$	associatividade de $\circ_T$

Leia assim As leis são as da soma, $a + 0 = a$, $0 + a = a$, $(a + b) + c = a + (b + c)$, com η no papel do 0.
Categoria = setas que se compõem, com identidades, de forma associativa.

Proposição 1

Toda mônada satisfaz essas leis: elas *decorrem* da funtorialidade de T e da naturalidade e coerência de η, μ .

Proposição 2

As três leis valem $\iff \mathcal{C}_T$ é uma categoria, com $\text{Hom}(A, B) := \text{Hom}_{\mathcal{C}}(A, TB)$, composição $\circ_T$ e identidades η .

As leis do *bind* são os axiomas de uma categoria

lei do <i>bind</i>	em Set	axioma de $\mathcal{C}_T$
$f^* \circ \eta_A = f$	$\eta(x) \gg f = f(x)$	identidade à direita
$(\eta_A)^* = \text{id}_{TA}$	$m \gg \eta = m$	identidade à esquerda
$(g^* \circ f)^* = g^* \circ f^*$	$(m \gg f) \gg g = m \gg (\lambda x. f(x) \gg g)$	associatividade de $\circ_T$

Leia assim As leis são as da soma, $a + 0 = a$, $0 + a = a$, $(a + b) + c = a + (b + c)$, com η no papel do 0.
Categoria = setas que se compõem, com identidades, de forma associativa.

Proposição 1

Toda mônada satisfaz essas leis: elas *decorrem* da funtorialidade de T e da naturalidade e coerência de η, μ .

Proposição 2

As três leis valem $\iff \mathcal{C}_T$ é uma categoria, com $\text{Hom}(A, B) := \text{Hom}_{\mathcal{C}}(A, TB)$, composição $\circ_T$ e identidades η .

Programar com *bind* é compor morfismos em $\mathcal{C}_T$.

Perguntas possíveis: matemática

- **Uma frase para definir mônada?** Um monoide na categoria dos endofuntores: μ é o produto, η a unidade, e as leis são associatividade e unidade.
- **Por que exigir $|S| \geq 2$ e $E \neq \emptyset$?** Com um só estado, $T \cong \text{Id}$; com $E = \emptyset$, $U \cong \text{Id}$. Nesses casos a lei trivial existe.
- **Então $(A \times S)^S + E$ não é mônada?** O resultado exclui leis distributivas; não afirma que esse funtor não admita nenhuma estrutura de mônada.
- **Os resultados são novos?** O trabalho é expositivo: a correspondência entre *bind* e Kleisli é clássica (Manes, 1976; Moggi, 1991); a contribuição é reunir a teoria com demonstrações explícitas.
- **De onde vem a ideia?** Na matemática, Godement (1958), Eilenberg–Moore e Kleisli (1965); em computação, Moggi (1989, 1991) e Wadler (1992, 1995).

Perguntas possíveis: programação

- **Onde se usa isso?** Em Haskell, `IO`, `Maybe`, listas e `State` são mônadas; a notação `do` é açúcar sintático para o `bind`.
- **Por que Haskell chama $\eta(x) \gg= f = f(x)$ de identidade à esquerda?** Porque `return` fica à esquerda do `>>=`. Em $\mathcal{C}_T$ a mesma lei é $f \circ_T \eta = f$, identidade à direita: mesma lei, outra ordem de escrita.
- **Mas `StateT s (Either e)` existe!** Ele é $s \rightarrow \text{Either } e (a, s)$, isto é, $((A \times S) + E)^S$: o erro fica *dentro* da função. A impossibilidade é sobre $(A \times S)^S + E$, com o erro fora.
- **E os *monad transformers*?** Cada um acrescenta um efeito fixo a uma mônada base, como `StateT s m a = s -> m (a, s)`. Não é a composição de funtores, e nem toda mônada tem transformer (`IO` não tem).

Prova: as leis do *bind* decorrem da estrutura

$$f^* \circ \eta_A = \mu_B \circ T(f) \circ \eta_A = \mu_B \circ \eta_{TB} \circ f = f \quad (\text{natur. de } \eta; \text{ unidade externa})$$

$$(\eta_A)^* = \mu_A \circ T(\eta_A) = \text{id}_{TA} \quad (\text{unidade interna})$$

$$(g^* \circ f)^* = \mu_C \circ T(\mu_C) \circ T^2(g) \circ T(f) \quad (\text{functorialidade de } T)$$

$$= \mu_C \circ \mu_{TC} \circ T^2(g) \circ T(f) \quad (\text{associatividade de } \mu)$$

$$= \mu_C \circ T(g) \circ \mu_B \circ T(f) = g^* \circ f^* \quad (\text{natur. de } \mu)$$

Prova: bind equivale a $\mathcal{C}_T$ ser categoria

($\Rightarrow$) Para $f : A \rightarrow TB$, $g : B \rightarrow TC$, $h : C \rightarrow TD$:

- $f \circ_T \eta_A = f^* \circ \eta_A = f$ e $\eta_B \circ_T f = (\eta_B)^* \circ f = f$;
- $h \circ_T (g \circ_T f) = h^* \circ g^* \circ f = (h^* \circ g)^* \circ f = (h \circ_T g) \circ_T f$.

($\Leftarrow$) Se $\mathcal{C}_T$ é categoria:

- $f \circ_T \eta_A = f$ é a lei (L1);
- $\eta_B \circ_T f = f$ com $f := \text{id}_{TB} \in \text{Hom}_{\mathcal{C}_T}(TB, B)$ dá $(\eta_B)^* = \text{id}_{TB}$, a lei (L2);
- a associatividade com $\text{id}_{TA} \in \text{Hom}_{\mathcal{C}_T}(TA, A)$, f e g dá $g^* \circ f^* = g \circ_T (f \circ_T \text{id}_{TA}) = (g \circ_T f) \circ_T \text{id}_{TA} = (g^* \circ f)^*$, a lei (L3).

Reconstrução: uma tripla de Kleisli $(T, \eta, (-)^*)$ basta, com $T(f) := (\eta_B \circ f)^*$ e $\mu_A := (\text{id}_{TA})^*$.

Prova: o estado não distribui sobre a exceção

Usa-se só (ii') $\lambda_A \circ T(\eta_A^U) = \eta_{TA}^U$ e (iii') $\lambda_A \circ \mu_{UA}^T = U(\mu_A^T) \circ \lambda_{TA} \circ T(\lambda_A)$.

Passo 1. Se $k \in TUA$ só produz erros, $\lambda_A(k)$ é erro: $k = TU(!)(k_0)$ com $! : \emptyset \rightarrow A$; por naturalidade $\lambda_A(k) = UT(!)(\lambda_\emptyset(k_0))$; como $S \neq \emptyset$, $T\emptyset = \emptyset$, logo $UT\emptyset = \emptyset + E$ e $\lambda_\emptyset(k_0)$ é erro.

Passo 2. $A = \{*\}$, $s_0 \neq s_1$, $w(s_0) = (\iota_1(*), s_0)$, $w(s) = (\iota_2(e), s)$ se $s \neq s_0$.

- **Caso** $\lambda_A(w) = \iota_2(e')$. $z := \lambda s. (w, s_0)$. Então $\mu_{UA}^T(z) = T(\eta_A^U)(c)$, $c = \lambda s. (*, s_0)$; por (ii') o lado esquerdo de (iii') é *sucesso*. Mas $T(\lambda_A)(z)$ só produz erros e, pelo Passo 1, o lado direito é *erro*.
- **Caso** $\lambda_A(w) = \iota_1(h)$. $z := \lambda s. (w, s_1)$. Então $\mu_{UA}^T(z)$ só produz erros, e o lado esquerdo é *erro* (Passo 1). Mas $T(\lambda_A)(z) = T(\eta_{TA}^U)(h')$ com $h' = \lambda s. (h, s_1)$, e por (ii') o lado direito é *sucesso*.

Em ambos os casos (iii') falha. ■

Reserva: definições compactas

Exceção (em qualquer categoria com coprodutos)

$$TA = A + E, \quad T(f) = f + \text{id}_E = [\iota_1 \circ f, \iota_2], \quad \eta_A = \iota_1, \quad \mu_A = [\text{id}_{A+E}, \iota_2] : \\ (A + E) + E \rightarrow A + E$$

$[u, v]$: definição por casos, a única seta com $[u, v] \circ \iota_1 = u$ e $[u, v] \circ \iota_2 = v$. $E = 1$ (um elemento): $A + 1$ é a *Maybe*.

Estado (em uma categoria cartesiana fechada, como Set)

$$TA = (A \times S)^S, \quad T(f)(h) = (f \times \text{id}_S) \circ h, \quad \eta_A(x) = \lambda s. (x, s), \quad \mu_A(F) = \text{ev} \circ F \\ \text{ev} : (A \times S)^S \times S \rightarrow A \times S, \text{ ev}(g, s') = g(s') : \text{rodar a próxima computação no estado novo. } \lambda s. e = a \\ \text{função } s \mapsto e.$$

São as mesmas definições do slide 10: lá, escritas por casos e por elementos.